

Overview:

The program is a multi-variable linear model that is trained on specific information about a movie to predict its Worldwide Gross. The returned β -coefficients for the model can then be used by the user to plug in the appropriate x's and try to find the Worldwide Gross of a movie based on the model's factors. The information for the model is based on a dataset that I got off Kaggle.

Data processing:

Before I put the dataset into Rust, I checked the linearity of a factor to Worldwide Gross in RStudio, which led me to remove some columns, one being a one-to-one copy of Worldwide Gross. I also deleted a few other columns that were causing an access os error, causing the program to fail in reading the full dataset. Ultimately, the final version of the dataset was an all-numerical dataset of 14 columns by 6,570 rows.

Code structure:

The code is split into two parts: a module called Read_Clean that takes a dataset, reads it in, and cleans the dataset appropriately so that it is prepared for the regression model; and the main file, which has two evaluation functions and the actual main function.

The module consists of 2 structs and 4 functions. The structs are called DirtyRecord and CleanRecord, which are used to clean the dataset by converting all values to strings in the DirtyRecord struct and then going and cleaning it using the CleanRecord struct.

- The first function is called `clean_record`, which uses the two structs to clean the dataset and handle missing values. It takes a DirtyRecord struct as input and outputs a CleanRecord struct.
- The second function is called `filter_clean_record`, which is a helper function that takes a vector of CleanRecord, iterates through it using `into_iter`, and filters out invalid CleanRecords where the Worldwide Gross is equal to 0 because those records are irrelevant to the model.
- The third function is called `process_csv_file`, which doesn't take an input and returns a vector of CleanRecord. This function reads in a CSV file, uses the `clean_record` function to clean the dataset after it is read in, and finally uses the `filter_clean_record` function to filter out irrelevant information before returning the vector.
- The fourth function is called `convert_to_xy`; this function takes in a vector of CleanRecord and a noise standard deviation, and returns an Array2 and Array1 as prep for the `train_test_split` call in main. This function adds all the desired factors to a vector and adds noise to Worldwide Gross, to avoid overfitting, before adding that to the vector as well. It then creates an Array2 out of that vector, creates a y variable which is an Array1 of Worldwide Gross, and an x variable which is an Array2 of all the factors I want to train the model on.

The main file consists of three functions.

- The first function is called `calculate_r2_score`, which takes in an Array1 of predictions and an Array1 of the actual values and returns a float of how well the model fits the data. The function calculates the total variance of the actual values, then calculates the residual sum of squares for the predictions, and uses those calculations for the R^2 calculation.

- The second function is called `calculate_rmse`, which calculates the root mean squared error with its inputs being an `Array1` of predictions and an `Array1` of actual values, and returns the RMSE as a float. The function calculates the sum of squared error of the predictions and returns the square root of that value over the length of the actual values array.

The main function calls the `process_csv_file` to get the cleaned dataset, defines the noise standard deviation before calling the `convert_xy` function, whose output is used in the `train_test_split` call. The model is then created from the training data, and the coefficients, intercepts, and evaluation calculations are printed.

Tests:

```
running 3 tests
test tests::test_calculate_r2_score_perfect_fit ... ok
test Read_Clean::tests::test_clean_record_basic ... ok
test tests::test_calculate_r2_score_zero_fit ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

The first test checks that the R-squared calculation is one when the predictions perfectly match the actual data, which is important because if it is a one-to-one match, then it is very likely that the model is overfitting. So it is important to make sure the function works.

The second test is in the module, and its purpose is to check that a dirty record is properly cleaned and turned into a clean record. This is important because if the data is not properly cleaned, it can lead to errors or miscalculated values.

The third test is another check to make sure the R-squared is calculated correctly for a very low value, which is important because it means that the model is underfitting.

Results:

```
Coefficients: [0.033236369496935377, 18853651615416.047, -18853651615415.055, 1.0029553714863337, 1648074783073.62, 1648074783073.456, -1648074783073.567, -0.016894689516422885, -7535.614229726296, 0.003111234412529225], shape=[10], strides=[1], layout=CFCf (0xf), const ndim=1
Intercept: 41059.04439487901
Root Mean Squared Error (RMSE): 41670731.72767182
R^2 Score: 0.9464851177258946
```

The coefficients are what the corresponding factors' value would be calculated by, along with the intercept added to get a predicted Worldwide Gross value. The RMSE means that the model is deviating by about 41.6 million off the actual value, which is decent because the Worldwide Gross values are operating in the hundreds of millions up to a couple of billion at times. The R-squared value means that the model is accounting for roughly 94.6 percent of the data, which is a very good fit and means the model is operating very well.

Instructions:

To build and run the code, all you have to do is go into the module and change the desired file name in the `process_csv_file` function (making sure that the new file has the appropriate number of columns and correct column labels) and then simply run the program. The run time for the program is very fast if it has already loaded all the dependencies, and if it hasn't, it is only a couple of seconds.